

Rastro.

O caça-bug metódico para quem constrói software com IA. Em vez de **chutar correção** quando o código quebra, o Rastro segue o rastro do erro até a **causa-raiz** — e para o loop que queima a sua assinatura.

O QUÊ

Depuração guiada

ONDE

Dentro do Claude Code

COMO

Você conversa, em português

01 O que é o Rastro

Todo mundo que constrói com IA conhece a cena: o código quebra, você (ou o próprio Claude) começa a testar correções no chute, mexe em coisa demais, e **cada conserto quebra outra coisa**. A assinatura vai embora num loop sem fim. O Rastro troca o chute pelo método: reproduzir o bug, observar o que acontece de verdade, achar a causa-raiz, corrigir o mínimo, conferir e blindar contra a volta do problema.

A LEI DE FERRO

Nenhuma correção antes de investigar a causa-raiz. Consertar o sintoma é falhar.

Estudos de campo: com método, um bug leva 15–30 min e acerta de primeira em ~95% das vezes. No chute, são 2–3 h de sofrimento e ~40%. Projeto tocado sem disciplina gasta de 4 a 6 vezes mais tokens.

02

Como instalar

Leva um minuto. Você não precisa saber programar para instalar — só descompactar e copiar uma pasta.

1

Descompacte o arquivo

Abra o `Rastro.zip` — dentro vem uma pasta chamada `Rastro` (com R maiúsculo).

2

Copie para dentro do seu projeto

Coloque a pasta `Rastro` dentro de `.claude/skills/` do seu projeto. Para usar em todos os projetos, use `~/ .claude/skills/`.

3

Abra o Claude Code e chame

Na pasta do seu projeto, é só pedir para depurar um bug. Na primeira vez, o Rastro faz um setup rápido (veja o próximo passo).

03

A primeira conversa

SETUP DE 1 MINUTO

O Rastro se apresenta e aprende o seu projeto

Da primeira vez, ele pergunta o essencial — a linguagem do projeto, **o comando que roda os seus testes**, como você roda o app e onde ficam os erros. Saber "o teste que a IA pode rodar" é o que faz o método inteiro funcionar. Depois disso, a parte de configuração some sozinha e você nunca mais repete.

"Configura o Rastro aqui no meu projeto"

04

O dia a dia — os 6 modos

Você não decora comando nenhum. Só descreve o problema em português, ou cola a mensagem de erro. O Rastro escolhe o modo certo.

MODO 1 · O CORAÇÃO

Depurar

O caminho completo para um bug, do começo ao fim: reproduzir → achar a causa-raiz → corrigir o mínimo → conferir → blindar. Sem pular etapa e sem chutar.

VOCÊ DIZ "Tá dando erro quando eu salvo o formulário, me ajuda a achar a causa."

O que esperar: ele reproduz o erro na sua frente, mostra onde o dado primeiro fica errado, propõe a menor correção possível e confirma rodando o seu teste.

MODO 2

Reproduzir

Transforma "às vezes quebra" em passos fixos e num teste que a IA consegue rodar sozinha.

"não sei quando dá esse bug, me ajuda a reproduzir"

MODO 3

Causa-raiz

As ferramentas para achar o porquê, não só o quê: os "5 porquês" e rastrear o erro até a origem.

"vamos investigar a causa de verdade dessa vez"

MODO 4

Sair do loop

A emergência: quando cada conserto quebra outra coisa, ele para tudo, reverte e reseta o raciocínio.

"a IA tá em loop e quebrando tudo, para isso"

MODO 5

Blindar

Confere de verdade, cria uma trava para o bug nunca voltar e limpa qualquer sobra de teste no código.

"garante que esse bug não volta mais"

MODO 6 · A MEMÓRIA

Diário

Cada bug resolvido fica guardado no seu computador — sintoma, causa e correção. Na próxima vez que um erro parecido aparecer, o Rastro reconhece e você não gasta tokens redescobrimo o que já sabia.

VOCÊ DIZ "já vi esse erro antes?"

O que esperar: ele responde "já vi algo parecido" com a causa e a correção antigas, ou avisa que é um bug novo. É a economia de token que **compõe com o tempo**.

05

As regras de ouro

- 1 Nunca conserta no escuro.** Sem reproduzir e entender a causa, não muda o código.
- 2 Observa, não adivinha.** O código diz o que deveria fazer; só rodando se vê o que ele faz.
- 3 Uma mudança por vez.** A menor possível — nada de "já que estou aqui".
- 4 Nunca empilha patch.** 2–3 falhas? Reverte tudo e questiona a arquitetura.
- 5 Nunca inventa.** Só o que rodou de verdade — dado, log, erro e comportamento reais.
- 6 Tudo no seu computador.** Nada do seu código sai para fora; você aprova cada correção.

06

Quando não usar

Para estes casos, resolva direto — não precisa de método:

- ✗ Erro de sintaxe ou de digitação óbvio: o próprio editor já aponta onde é.
- ✗ "Onde está tal coisa definida?" — isso é uma busca no código, não uma depuração.
- ✗ Não é bug, é uma funcionalidade que ainda falta — isso é planejar, não depurar.

Rastro.

Requer Claude Code + Python 3 · offline · sem chave de API